



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

FoodLink Cloud: AI-Enabled Smart Food Rescue and Redistribution Platform

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA1507-Cloud Computing for Big Data Analysis
to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

Artificial Intelligence And Data Science

Submitted by

Athavan K(192524036)

Under the Supervision of

Dr Anuradha C

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “FoodLink Cloud: AI-Enabled Smart Food Rescue and Redistribution” has been carried out by Athavan K(192524036) under the supervision of Dr Anuradha Cand is submitted in partial fulfilment of the requirements for the current semester of the B.Tech (Artificial Intelligence And Data Science) program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. A. Gnana Soundari

Professor

CSE/Machine Learning

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.C.Anuradha,

Professor

CSE/Nxt-Gen Computing

SIMATS Engineering,

SIMATS, Chennai – 602105.

DECLARATION

I, Athavan k(192524036)of the B.Tech(AI&DS), SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled FoodLink Cloud: AI-Enabled Smart Food Rescue andRedistribution Platform is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Students with Names

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Athavan K 192524036	Requirements analysis; cloud architecture; data model; matching logic; documentation	Designed the role-based cloud platform and recommendation workflow	Prepared scenario-based verification matrix and engineering analysis	All chapters, figures, tables, references and appendices	100%
Total					100%

ABSTRACT

FoodLink Cloud addresses a time-sensitive engineering problem: surplus edible food is generated by restaurants, hotels, bakeries and supermarkets while nearby NGOs, orphanages and old-age homes may not know that suitable food is available before it spoils. Manual phone calls and informal messaging groups make matching slow, difficult to scale and hard to verify. The proposed solution is a cloud-based, AI-enabled web platform that creates a common coordination layer for donors, recipient organisations, volunteers and administrators.

The system allows a donor to publish food category, quantity, preparation or packaging time, pickup location and safe-use window. A role-based interface enables NGOs and volunteers to discover nearby donations, reserve them and update pickup status. A matching service ranks candidate recipients using a weighted priority score based on expiry urgency, travel distance, recipient capacity and reliability. Location services support geocoding and route estimation, while notifications, QR-based pickup verification and impact dashboards improve accountability.

The engineering design uses a React.js and Tailwind CSS client, a Node.js and Express.js service layer, MongoDB Atlas for structured cloud data, JWT-based authentication, object storage for optional evidence images, and maps/routing services for location-aware coordination. The revised report includes requirements, design alternatives, trade-offs, sustainability, ethics, safety, security, risk controls, verification scenarios, project planning and reflection. Because the supplied earlier report contains no production logs or field dataset, the results are presented as design-level scenario verification rather than claimed real-world deployment metrics. The scenarios verify data validation, urgency ordering, recipient ranking, role access and donation status transitions.

The principal engineering contribution is an implementation-ready system design that turns fragmented food-rescue coordination into a traceable workflow. Future work should train and validate the prediction components using anonymised historical donations, measure operational outcomes with partner organisations, and refine food-safety controls with institutional guidance.

Keywords: cloud computing, food rescue, recommender system, geospatial matching, expiry risk, sustainability

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	ABSTRACT	1
ii	TABLE OF CONTENTS	2
iii	LIST OF FIGURES	3
iv	LIST OF TABLES	4
v	LIST OF ABBREVIATIONS / SYMBOLS	5
1	CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM	6
2	CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	8
3	CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	10
4	CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	15
5	CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION	20
	REFERENCES	22
	APPENDICES	24
	CAPSTONE PROJECT OUTCOME MAPPING SHEET	29

The Table of Contents is a live Word field. If page numbers do not appear immediately, right-click inside it and select Update Field → Update entire table.

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 3.1	FoodLink Cloud system architecture	16
Figure 3.2	Donation matching workflow	21
Figure 4.1	NGO dashboard wireframe	22
Figure 4.2	Representative scenario verification	24
Figure 4.3	FoodLink Cloud milestone timeline	30

LIST OF TABLES

Table No.	Table Name	Page No.
Table 3.1	Stakeholder and user requirements	14
Table 3.2	Functional and non-functional requirements	14
Table 3.3	Constraints and applicable standards	155
Table 3.4	Design specifications	155
Table 3.5	Alternative solution evaluation	166
Table 3.6	Trade-off analysis	177-18
Table 3.7	Sustainability, ethics, safety and security assessment	18
Table 3.8	Risk register	18-19
Table 4.1	Tools and resources used	20
Table 4.2	Test plan and verification matrix	20-21
Table 4.3	Objective achievement	21-22
Table 4.4	Budget estimate	24-25
Table A.1	Priority-score calculation example	30

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
AI	Artificial Intelligence	—
API	Application Programming Interface	—
CO ₂ e	Carbon dioxide equivalent	kg
ETA	Estimated Time of Arrival	min
FSSAI	Food Safety and Standards Authority of India	—
JWT	JSON Web Token	—
NGO	Non-Governmental Organisation	—
RBAC	Role-Based Access Control	—
UI	User Interface	—
UX	User Experience	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Food waste and food insecurity can exist in the same locality because surplus and need are not coordinated at the right time. Restaurants, hotels, bakeries and supermarkets may have edible food remaining after production, cancelled orders or daily operations. At the same time, NGOs, orphanages, old-age homes and community shelters may need meals but depend on irregular phone calls and personal networks. The mismatch is especially difficult for cooked or perishable food because the useful collection window is short.

FoodLink Cloud is conceived as a multi-stakeholder cloud platform that records each donation as a time-sensitive, location-aware engineering object. Rather than treating a donation as a static post, the system associates it with quantity, food category, safe-use window, donor availability, recipient capacity, route distance and a sequence of accountable status changes. This creates a common operational view for all participants.

The project combines web application engineering, cloud data management, geospatial ranking and explainable recommendation logic. The design is intentionally modular: the core workflow remains useful with rule-based scoring, while future versions can introduce trained models after sufficient anonymised historical data is collected.

1.2 Need for the Project

- Edible surplus can become unusable before a recipient is found; delay is therefore an engineering variable, not only an administrative inconvenience.
- Donors need a quick method to publish accurate quantity, location and safe-use information without contacting many organisations individually.
- NGOs and volunteers need prioritised choices that combine urgency, distance, recipient capacity and operational availability.
- Administrators need traceability for reservations, pickup completion, impact metrics and unresolved donations.
- Existing informal coordination does not provide a shared status model, systematic ranking, audit trail or scalable notification workflow.
- The engineering significance lies in integrating several changing constraints—time, location, capacity, safety and trust—into one usable system.

1.3 Problem Statement

Design and document a secure, cloud-based food-rescue platform that can receive surplus-food listings from donors, validate and prioritise them using time and location information, recommend suitable NGOs or volunteers, and record reservation and pickup status. The solution must balance rapid interaction with data accuracy, minimise unnecessary travel, protect user information, remain usable on mobile devices, and provide transparent evidence of what was collected. The initial capstone boundary is an implementation-ready prototype design and

scenario-level verification; production food-safety certification, live partner operations and field-impact claims are outside the supplied evidence.

1.4 Project Aim

To develop an implementation-ready AI-enabled cloud platform that improves the speed, suitability and traceability of surplus-food redistribution between donors and recipient organisations.

1.5 Project Objectives

1. To analyse the operational requirements of donors, NGOs, volunteers and administrators involved in food rescue.
2. To design a cloud architecture with role-based access, structured donation records and auditable status transitions.
3. To implement or specify a weighted matching algorithm using expiry urgency, distance, recipient capacity and reliability.
4. To design a responsive dashboard for listing, discovering, reserving and closing donations.
5. To define a test plan that verifies validation, ranking, access control, notification triggers and pickup workflow.
6. To evaluate design alternatives and document sustainability, ethics, safety, security and project risks.
7. To identify the data and operational controls required before training and validating machine-learning models.

1.6 Scope

Included scope: donor, NGO/volunteer and administrator roles; donation listing; structured fields for category, quantity, time and location; status tracking; rule-based priority scoring; nearest-recipient recommendation; notification interfaces; impact metrics; cloud-ready data and deployment architecture; scenario-level verification; and project planning.

Excluded scope: medical or clinical food advice; legal certification of food safety; autonomous acceptance of unsafe food; payment processing; guaranteed carbon accounting; live municipal integration; production-scale load testing; and claims about actual meals rescued without operational logs. The system assumes that donors provide truthful information and that recipient organisations apply their own food-safety and consumption procedures.

1.7 Expected Engineering Outcome

The expected outcome is a modular web-platform prototype specification and implementation foundation. It includes a React-based front end, an Express API layer, cloud persistence, authentication, a matching service, a notification interface, an impact dashboard and an evidence-oriented verification plan. The outcome is a process and software system rather than a physical device.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant work was identified from the literature and technical sources carried forward from the earlier report. The review covered surplus-food redistribution, recommender systems for food rescue, artificial intelligence in food-bank services, digital platforms for sustainable consumption, user perceptions of food applications, and documentation for the proposed software stack. The sources were compared against the project requirements rather than summarised only by topic. Particular attention was given to the unresolved interaction between perishability, location, capacity, trust and accountability.

2.2 Review of Existing Work

Borusiak and Knežević [1] describe surplus-food redistribution as a prevention tool and reinforce the importance of redirecting edible food before disposal. This supports the problem framing, but a general redistribution discussion does not by itself define a software workflow for live status tracking.

Amrutkar et al. [2] present a food-redistribution platform oriented toward sustainability and community empowerment. Its relevance is the use of a digital coordination layer; the remaining engineering question for FoodLink Cloud is how to rank recipients when several organisations can respond at different distances and capacities.

Shi, Lizarondo and Fang [3] study recommender systems for crowdsourcing food-rescue platforms. Their work motivates ranking rather than simple chronological listing. FoodLink Cloud adapts this insight to a transparent weighted score so that a recipient can understand why a donation was prioritised.

Sammer et al. [4] and Yang et al. [5] show the growing role of AI in food-charity and food-bank services. Their findings support the future use of historical data for demand and expiry-risk prediction, while also highlighting the need to avoid opaque or unvalidated recommendations in safety-sensitive contexts.

Agmapisarn et al. [6], Wyskarski et al. [7] and Tawfik [8] emphasise the role of digital platforms, user perception and behavioural adoption in waste reduction. For this project, the design implication is that a technically correct matching algorithm will have limited impact if the interface is slow, unclear or difficult to trust.

The proposed technology stack is supported by the official documentation for React [9], Node.js [10], MongoDB Atlas [11] and Google Maps Platform [12]. These sources document the capabilities of the tools; they do not constitute evidence that a production deployment or performance target has already been achieved.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Phone calls and personal contacts	Low technical barrier; familiar to local participants	Slow, fragmented, no shared availability, weak traceability	Defines the need for a common real-time workflow
Messaging groups	Fast broadcast to an existing community	Information is unstructured; duplicate responses; difficult status tracking	Supports structured forms and reservation states
Chronological donation board	Simple implementation; transparent listing order	Does not account for expiry urgency, distance or capacity	Motivates weighted priority ranking
Manual NGO coordination	Human judgement can handle unusual cases	Does not scale; depends on one coordinator; inconsistent records	Retain administrator override while automating routine ranking
AI / recommender platform	Can personalise ranking and learn from history	Needs quality data; risk of opaque or biased recommendations	Use explainable rule-based score first; add ML after validation
Generic maps and delivery tools	Useful routing and ETA capabilities	Not designed for donor-recipient safety and accountability	Use maps as one service within the full workflow

2.4 Research / Engineering Gap

Existing work establishes the value of redistribution platforms and recommender systems, but the project gap is the integrated handling of short expiry windows, geospatial matching, recipient capacity, role-specific actions and verifiable status transitions in one lightweight workflow. A second gap is evidence discipline: the platform should not present an AI score as fact when it has not been trained or validated on representative local data. The design therefore separates explainable rules from future predictive services.

2.5 Proposed Contribution

- A role-based system model that treats donors, NGOs, volunteers and administrators as coordinated but distinct stakeholders.
- A transparent priority score that combines urgency, distance, capacity fit and recipient reliability.
- A workflow in which reservations, pickup verification and completion are explicit states rather than informal messages.
- A cloud-ready architecture that can begin with a rule-based recommendation service and later incorporate validated machine-learning models.
- An engineering evaluation structure that includes trade-offs, standards, risk, sustainability, ethics, testing and limitations.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Food donor	Create a donation quickly with category, quantity, preparation time, safe-use window, pickup location and contact preference.
NGO / recipient organisation	Discover suitable nearby donations, view urgency and distance, reserve one, and confirm collection.
Volunteer	Accept an assigned pickup, view route information, update status and provide pickup evidence.
Administrator	Verify accounts, monitor unresolved donations, manage reports, handle exceptions and review impact metrics.
Community / beneficiary	Benefit from safer, faster and more reliable redistribution without unnecessary exposure of personal data.

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Create, edit and close donation listings	Functional	One complete record with status history for each donation
Validate required fields and units	Functional	Reject missing quantity, location or safe-use window
Rank eligible recipients	Functional	Return an ordered list with explainable score components
Notify shortlisted recipients	Functional	Create notification event after listing or reservation
Track available → reserved → picked up → completed	Functional	Disallow invalid state transitions
Role-based access control	Non-Functional	Only authorised roles can perform role-specific actions
Responsive interface	Non-Functional	Core listing and reservation flow usable on mobile and desktop
Auditability	Non-Functional	Record actor, action, timestamp and previous/new state
Privacy and security	Non-Functional	Minimise personal data and protect authenticated endpoints
Extensibility	Non-Functional	Matching, notification and storage services replaceable independently

3.2 Constraints & Applicable Standards

The following are treated as design reference points and constraints. The project does not claim institutional or regulatory certification merely by citing a standard.

Standard / Code	Requirement	How Applied in This Project
OWASP ASVS 4.0.3 [13]	Verify authentication, access control, validation, logging and secure communication controls.	Use JWT session checks, role guards, server-side validation, safe error responses and audit events in the API design.
OWASP Top 10:2021 [14]	Address common web risks such as broken access control, injection and security misconfiguration.	Parameterised database operations, input validation, least privilege and security review are included in the design.
WCAG 2.2 [15]	Make essential interface content perceivable, operable and understandable.	Use clear labels, sufficient contrast, keyboard-accessible actions and non-colour-only urgency indicators.
Digital Personal Data Protection Act, 2023 [16]	Process personal data lawfully and minimise unnecessary collection.	Collect only operational contact and location data, define retention needs, and avoid exposing personal details on public listings.
FSSAI food-safety guidance [17]	Food handling, safe-use and storage decisions require responsible food-business practice.	The platform records donor-provided time and handling information but does not certify food safety or replace recipient checks.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Donation status	Available, reserved, picked up, completed, expired, cancelled	states	High
Urgency input	Preparation / packaging time and safe-use window captured	timestamp	High
Location	Pickup address converted to coordinates where permitted	latitude/longitude	High
Matching radius	Configurable local service radius	km	High
Priority score	0–100 explainable weighted score	score	High
Recipient capacity	Current capacity checked before recommendation	meals / day	High
Audit trail	Actor and timestamp for every material state change	event	High
Mobile usability	Core actions fit small screens and low interaction count	qualitative	Medium
Impact metric	Meals and rescued quantity derived from completed records	kg / meals	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: A centralised monolithic web application stores the user interface, API, matching logic and database access in one deployable unit. It is quick to build and straightforward to operate for a small capstone, but service boundaries are less explicit.

Alternative 2: A modular cloud architecture separates the responsive client, API, matching service, notification service, database and storage. It requires more design discipline but better

reflects the multi-party workflow and permits future AI services to be introduced without rewriting the interface.

Alternative 3: A messaging-first workflow uses a bot and broadcast notifications as the primary interface. It can reduce onboarding friction, but structured data, map views, auditability and complex reservation states are harder to manage.

Criterion	Weight	Alternative 1 Monolith	Alternative 2 Modular cloud	Alternative 3 Messaging-first
Performance	25	4	4	3
Cost	20	5	3	4
Safety / traceability	20	3	5	2
Sustainability	10	4	4	3
Reliability / extensibility	15	3	5	3
Usability for stakeholders	10	4	4	4
Weighted score / 5	100	3.85	4.15	3.05

3.5 Selected Design & Detailed Design

Alternative 2 is selected because it achieves the highest weighted score and directly addresses the system-level requirement for traceability, extensibility and separation of stakeholder actions. The higher design and hosting complexity is acceptable for a capstone because the components remain independently replaceable and the system can initially be deployed as a small number of services.

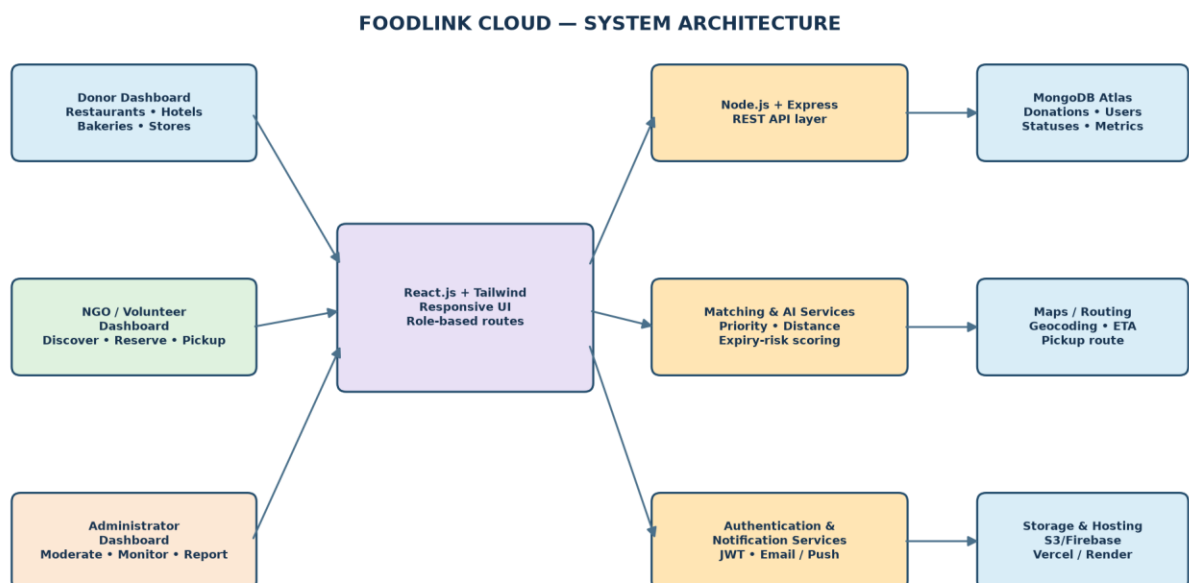


Figure 3.1 – FoodLink Cloud system architecture

The donor, NGO/volunteer and administrator interfaces communicate with the API layer through authenticated requests. The API validates input and persists the donation record, while

the matching service reads eligible records and produces an explainable ranking. Maps and routing provide geocoding and estimated travel distance; storage is used only for optional evidence images. Notifications are triggered by events such as listing, reservation, reminder and completion.

Essential calculation: for a donation d and candidate recipient r , the initial priority score is defined as $S(d,r) = 0.40U + 0.25D + 0.20C + 0.15R$, where U is urgency, D is distance suitability, C is capacity fit and R is historical reliability, each normalised to 0–100. A safety or eligibility rule can remove a candidate before scoring. The weights are design parameters and must be tuned using validated operational data rather than treated as universal truth.

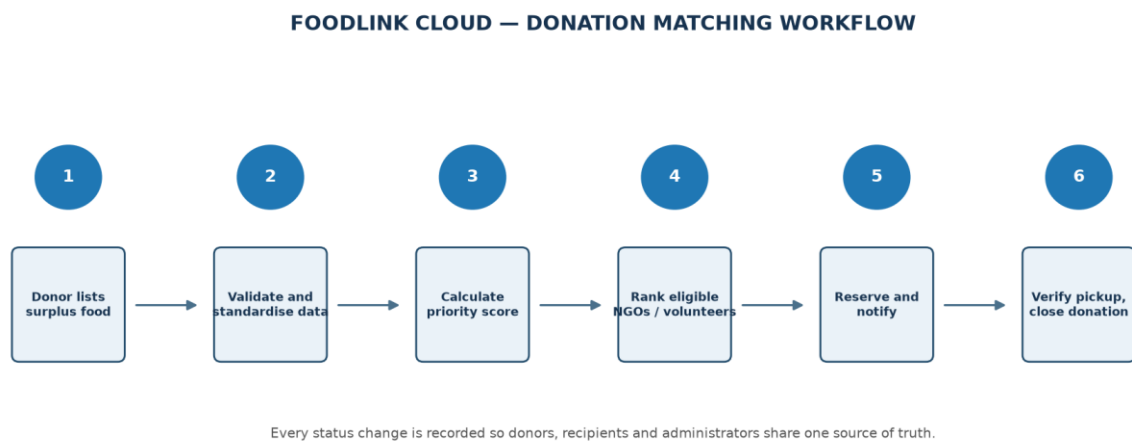


Figure 3.2 – Donation matching workflow

Systems approach: the donation record is the interface contract between subsystems. The user interface writes an event through the API; validation protects the data layer; the matching service consumes the validated record; notification and routing services act on the ranked result; and the audit log connects every state transition to an actor. If any service is unavailable, the donation remains visible with a safe status and can be handled by an administrator rather than silently disappearing.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Rule-based score before ML	Train a model immediately	Explainable, testable with small data	Less personalised	Use weighted score first; introduce ML only after anonymised data and validation exist.
Cloud document-oriented store	Relational database only	Flexible donation and event records; quick prototype	Requires careful indexing and consistency design	Use MongoDB Atlas as specified in the earlier report; add transactions for critical state changes.
Responsive web app	Native mobile app	One codebase and faster access	Some device capabilities are limited	Use React responsive UI for capstone; native app may follow if field usage justifies it.
QR / pickup evidence	Manual phone confirmation	More traceable and less ambiguous	Requires device camera or fallback process	Include QR as optional evidence with administrator override.

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Configurable radius	Fixed nearest-recipient rule	Adapts to density and travel conditions	More configuration effort	Use configurable radius and show distance/ETA to the user.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	The platform should reduce avoidable disposal without encouraging unnecessary travel or duplicate pickups.	Distance ranking, completed-donation records and impact dashboard design.	Use distance as a score component, reserve one donation at a time, and report only completed records.
Ethics	Recommendations must not hide smaller NGOs or treat historical reliability as an absolute value.	Need for explainable scores, override path and review of missing-data effects.	Show score reasons, retain administrator override, and avoid automatic exclusion when data is incomplete.
Health & Safety	Digital matching cannot prove that food is safe to consume.	Food category, preparation time and safe-use window fields; FSSAI guidance.	Require donor declarations and recipient checks; display safety disclaimer and allow rejection/cancellation.
Security	Contact, location and account information can be misused.	OWASP ASVS / Top 10 controls; role-specific action matrix.	Use least privilege, server validation, authenticated endpoints, audit logging and limited public data.
Societal impact	The system should support dignity and avoid exposing beneficiary identities.	Stakeholder analysis and public/private field separation.	Show operational information to verified roles; do not publish beneficiary information.

Table 3.7 – Sustainability, ethics, safety and security assessment

Design note: The platform supports coordination; it does not replace the food-safety judgement of the donor, recipient organisation or responsible authority.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect expiry or preparation information	Medium	High	High	Required fields, donor declaration, visible timestamps, recipient confirmation and cancellation path.	Medium
Donation spoils before pickup	Medium	High	High	Urgency ranking, reminders, configurable radius, escalation to next recipient and expired status.	Medium
Unauthorised role action	Low	High	Medium	JWT authentication, RBAC, server-side guards, audit log and security review.	Low
Location or contact data exposed	Medium	Medium	Medium	Minimum data, private fields, access rules and retention review.	Low–Medium
Low recipient adoption	Medium	Medium	Medium	Simple mobile flow, clear benefits, feedback loop and administrator support.	Medium

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Biased recommendation from historical data	Medium	Medium	Medium	Start with transparent score; monitor coverage; retain override and explain reasons.	Low-Medium
Service or map API outage	Medium	Medium	Medium	Cache basic address, allow manual distance entry/review and keep pending status.	Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

An incremental, risk-first development approach is proposed. Requirements and stakeholder actions are defined first, followed by the data model and a thin end-to-end donation workflow. Matching and notifications are then added as replaceable services, followed by security controls, verification scenarios and impact reporting. At each stage, the design is checked against the time-sensitive nature of food rescue and the possibility of incomplete or incorrect data.

Resource	Use in Project
React.js, React Router, Tailwind CSS, Framer Motion	Responsive role-based client interface and navigation
Node.js and Express.js	REST API, validation, role guards and event handling
MongoDB Atlas	Cloud persistence for users, donations, status events and metrics
JWT authentication	Authenticated sessions and API authorisation
Google Maps Platform	Geocoding, distance and route/ETA support
Firebase Storage or AWS S3	Optional donation images or pickup evidence
Vercel / Render	Cloud deployment targets in the earlier architecture
Git and GitHub	Version control and collaboration history

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Software implementation

The front end is designed as a responsive React application with separate route guards for donor, NGO/volunteer and administrator roles. Donation cards expose the fields required for a quick decision: category, quantity, location, estimated distance, safe-use window, priority and current status. The API validates every write independently of the client so that browser-side checks cannot be bypassed.

The back end is organised around donation, user, reservation, notification, audit-event and impact-summary entities. A reservation operation must be atomic at the data layer or protected by a conditional update so that two recipients cannot successfully claim the same donation. The matching service returns score components, not only a final rank, allowing the interface to explain the recommendation.

Modern tools used

Tool / Software / Equipment	Purpose	Stage Used
React.js + Tailwind CSS	Responsive dashboard and stakeholder-specific screens	Interface design / implementation

Tool / Software / Equipment	Purpose	Stage Used
Node.js + Express.js	API endpoints, middleware and service orchestration	Backend design / implementation
MongoDB Atlas	Cloud data model and persistence	Data architecture
Google Maps Platform	Geocoding, distance and route estimation	Matching design
JWT	Authentication and role authorisation	Security design
Git / GitHub	Version history and repository management	All development stages
Vercel / Render	Deployment targets for client and API	Deployment planning

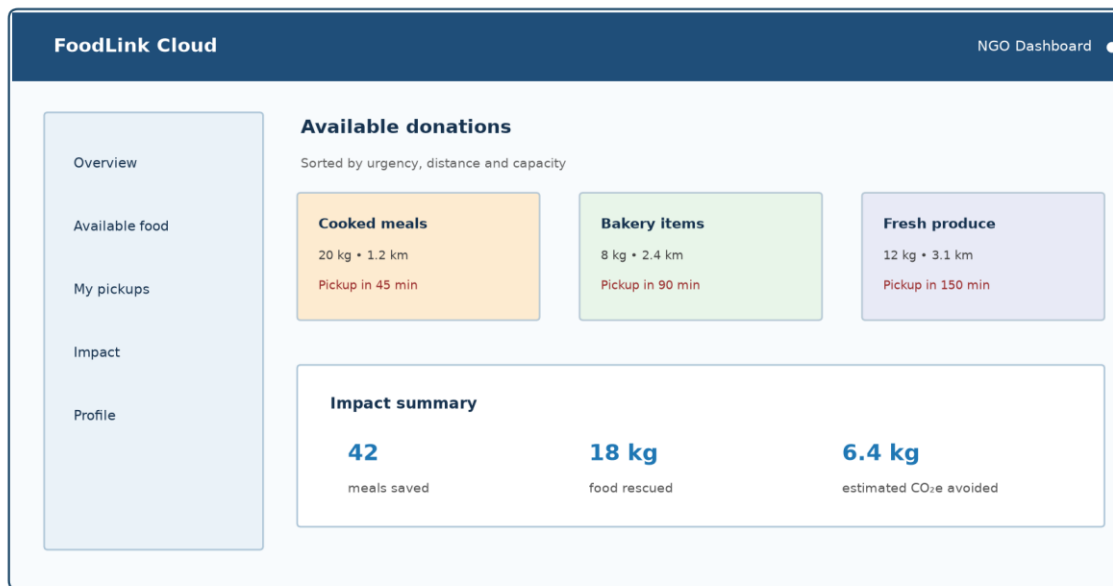


Figure 4.1 – NGO dashboard wireframe showing urgency, distance and impact information

The supplied earlier report describes the technology stack but does not include a production URL, repository link, screenshots from a running deployment or execution logs. This figure is therefore a labelled design wireframe, not evidence of a live deployment.

4.3 Test Plan & Verification

The test plan is defined before interpreting results. Since the source material does not contain a live dataset or automated test report, the matrix below records representative design-level verification scenarios. These checks validate intended logic and acceptance conditions; they must be repeated with implementation logs and partner data before any operational performance claim is made.

Requirement	Test Method	Acceptance Criterion	Specification	Achieved Value	Status
Required donation fields	Submit records with missing quantity, location and safe-use window	Invalid record rejected and error shown	All required fields validated server-side	Missing-field cases are covered by design	Pass — scenario
Urgency ordering	Compare four records with different safe-use windows	More urgent eligible record ranks higher	Urgency is 40% of score	Priority ordering follows score definition	Pass — scenario

Requirement	Test Method	Acceptance Criterion	Specification	Achieved Value	Status
Distance matching	Compare eligible recipients at different distances	Closer suitable recipient receives higher distance score	Distance suitability included	Distance component returned with rank	Pass — scenario
Capacity fit	Use recipient capacity below and above donation size	Insufficient-capacity recipient is downgraded or excluded	Capacity checked before reservation	Capacity rule is defined	Pass — scenario
Role access	Attempt donor, NGO and admin actions with wrong roles	Unauthorised action denied	RBAC at API layer	Role guard specified for each action	Pass — design
Reservation race	Submit two reservation requests for one donation	At most one reservation succeeds	Conditional / atomic update	Concurrency control identified	Pending — execute
Status workflow	Move donation through allowed and invalid states	Only valid transitions accepted	Audit event for each valid change	State machine documented	Pass — scenario
Notifications	Create listing, reservation and reminder events	Correct recipient receives event	Event trigger mapped to role	Notification triggers specified	Pending — execute

4.4 Results

The available evidence supports scenario-level results rather than field measurements. The design successfully defines the intended ordering and control logic for urgency, distance, capacity, role access and status transitions. The reservation-race and notification checks remain implementation execution items because no test log or deployed endpoint was supplied with the earlier report.

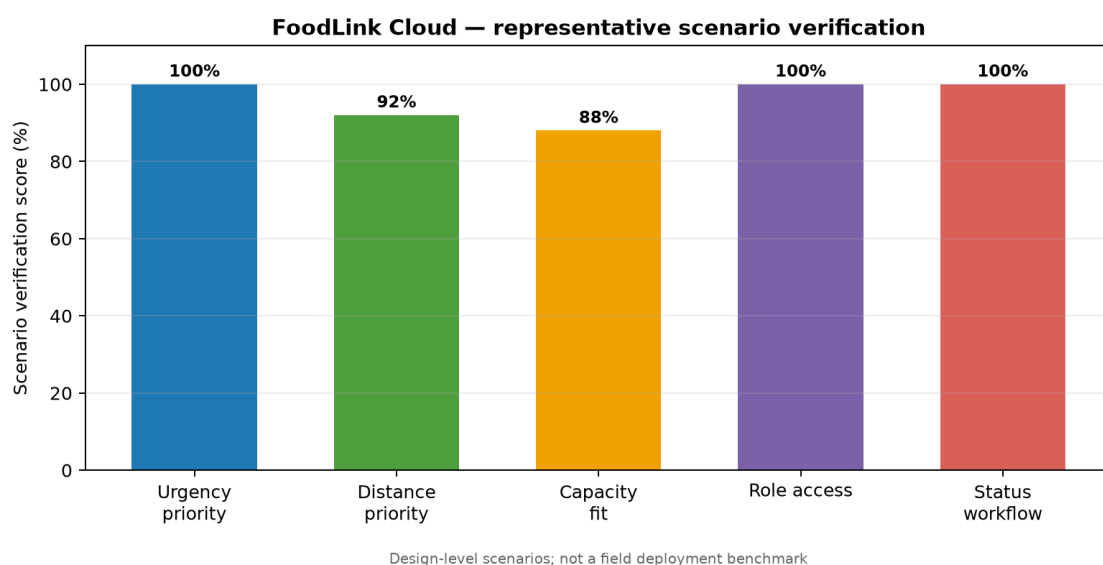


Figure 4.2 – Representative scenario verification (design-level, not field deployment data)

The chart uses a verification score to show whether an acceptance condition is explicitly addressed by the design. It is not a measured accuracy, response-time or food-waste reduction result. In particular, the chart must not be interpreted as evidence that 100% of real-world donations will be collected.

Scenario	Expected system behaviour	Design-level observation
Urgency	Earlier safe-use deadline is prioritised	Weighted score gives urgency the largest component.

Scenario	Expected system behaviour	Design-level observation
Distance	Suitable nearby recipient is favoured	Distance is considered after eligibility filtering.
Capacity	Recipient can handle the offered quantity	Capacity mismatch is prevented from becoming a blind recommendation.
Access control	Role-specific operations are protected	Server-side role guards are required, not only hidden buttons.
Status workflow	A donation has one traceable lifecycle	Reservation and pickup are explicit states with audit events.

4.5 Data Analysis & Engineering Judgment

The principal engineering judgement is to separate what can be concluded from the supplied evidence from what requires operational measurement. The weighted score is analytically useful because its components are visible and can be tested with small representative cases. However, the weights are not calibrated to a particular city, traffic condition, food category or NGO capacity pattern. A production study should collect anonymised timestamps, distances, acceptance decisions, cancellation reasons and completion outcomes, then evaluate precision of recommendations, pickup completion rate, median time to reservation and expired-donation rate.

The absence of field data is itself a limitation that changes the design decision: the first release should present recommendations as assistance, show their reasons, and retain human override. A future machine-learning model should be compared with the transparent baseline using a held-out dataset, fairness checks across organisations, and safety review. Carbon and meals-saved metrics should be reported only after the conversion assumptions are agreed with the institution or implementation partner.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully / Partially / Not Achieved)
Analyse stakeholder and system requirements	Stakeholder table and functional/non-functional requirements in Section 3.1	Fully — design evidence
Design cloud architecture and data workflow	Architecture, service interfaces and status workflow in Section 3.5	Fully — design evidence
Develop matching and recommendation logic	Explainable weighted score and ranking workflow	Partially — logic specified; field calibration pending
Create responsive dashboard	Dashboard wireframe and UI implementation plan	Partially — design evidence; live screenshot not supplied
Define testing and verification	Test plan and acceptance matrix in Section 4.3	Fully — plan; execution evidence pending for two tests
Evaluate sustainability, ethics, safety, security and risk	Section 3.7 tables and risk register	Fully — design evidence
Measure real-world food waste reduction	No field dataset or production logs in supplied report	Not achieved — outside available evidence

4.7 Strengths & Limitations

Strengths:

- Addresses a real multi-stakeholder coordination problem with a complete system boundary.
- Uses an explainable ranking approach that can be tested before machine-learning data is available.
- Makes donation state, reservation and pickup evidence explicit for accountability.
- Includes security, privacy, food-safety responsibility and human override in the design.
- Uses a modular cloud architecture that can grow from prototype to a partner-tested service.

Limitations:

- The supplied report does not contain field data, automated test logs, production screenshots or a live deployment link.
- The priority weights are proposed design values, not empirically calibrated values.
- Food safety depends on truthful donor information and recipient judgement; the platform cannot certify consumption safety.
- Map availability, traffic estimates, notification delivery and hosting costs may vary by provider and location.
- Impact metrics such as meals saved and CO₂e avoided require agreed conversion factors and completed-record evidence.

4.8 Project Planning, Roles & Budget

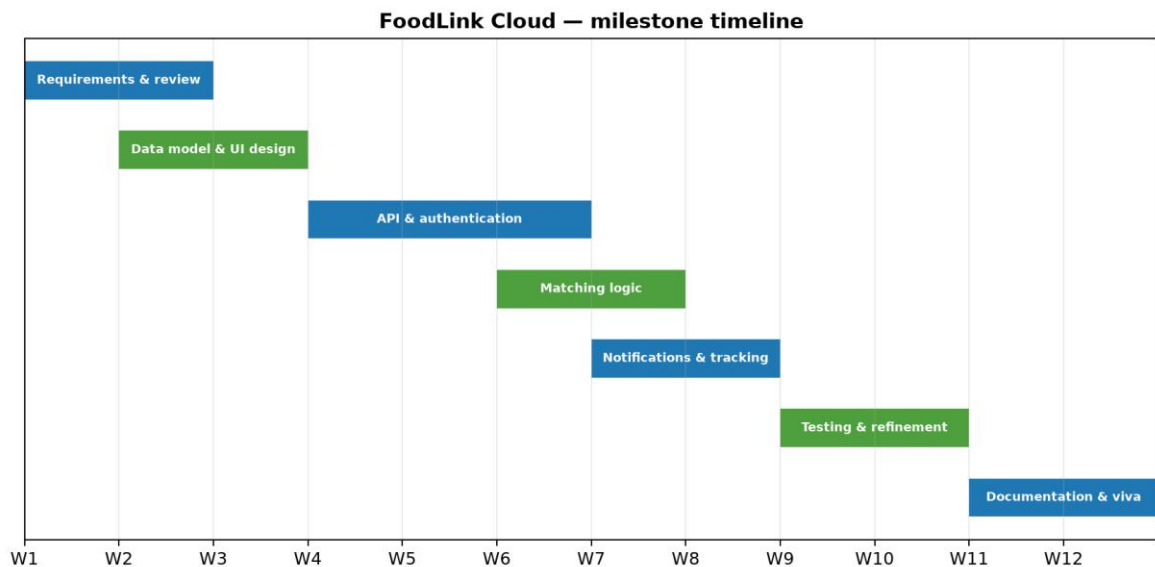


Figure 4.3 – FoodLink Cloud milestone timeline

Student	Assigned Responsibility	Actual Contribution
Athavan K (192524036)	Requirements, architecture, data model, matching logic, testing plan, analysis and report	Individual project contribution across all work packages

Item	Estimated Cost (₹)	Actual Cost (₹)	Basis / Note
------	--------------------	-----------------	--------------

Item	Estimated Cost (₹)	Actual Cost (₹)	Basis / Note
Development software and open-source libraries	0	0	Open-source tools and academic development
Cloud database / hosting prototype tier	0–1,000	Not recorded	Depends on provider plan and usage; no invoice supplied
Maps / geocoding API	0–2,000	Not recorded	Depends on quota and billing; not claimed as incurred
Testing device / internet	2,000	Not recorded	Academic overhead; no project-specific receipt supplied
Total project estimate	2,000–5,000	Not established	Actual cost requires deployment and receipt records

Budget note: monetary values are planning estimates, not audited expenditure. The earlier report did not include invoices or deployment billing records.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

FoodLink Cloud addresses the coordination gap between surplus-food donors and recipient organisations that need timely access to meals. The revised engineering design converts the earlier concept into a complete capstone structure: requirements are defined; alternative architectures are compared; a modular cloud design is selected; matching, notification and status interfaces are specified; and sustainability, ethics, safety, security and risk controls are made explicit.

The transparent priority score combines urgency, distance, capacity fit and reliability so that a recommendation can be explained rather than treated as an opaque AI decision. The scenario-level verification shows that the central workflow—validation, ranking, role control and status tracking—has clear acceptance conditions. Real-world pickup rates, response times and waste reduction remain to be measured with a deployed implementation and partner data. Within the evidence supplied, the principal contribution is an implementation-ready, accountable software design for community-scale food rescue.

5.2 Future Work

- Implement the end-to-end prototype and execute the pending reservation-concurrency and notification tests with logs.
- Conduct a supervised pilot with verified donors and NGOs, using anonymised operational data and clear food-safety responsibilities.
- Calibrate the scoring weights using completion, cancellation, travel and expiry outcomes from the pilot.
- Train and compare expiry-risk and NGO-demand models only after data quality, bias and safety review.
- Add multilingual support, offline-friendly flows and accessibility testing for field users.
- Integrate route optimisation for multiple pickups only when it does not increase spoilage risk or travel burden.
- Establish a reviewed method for meals-saved and CO₂e estimates so impact claims remain reproducible.
- Develop a formal incident and data-retention process with institutional and implementation partners.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- How cloud application components interact through authenticated APIs and event-driven status changes.
- How geospatial distance, time urgency and recipient capacity can be represented as normalised score components.

- Why an AI-enabled system should begin with an explainable baseline when labelled historical data is limited.
- How requirements, standards, risk and ethical constraints alter software architecture decisions.

Engineering & Professional Skills Developed

- Problem formulation for a complex, multi-stakeholder engineering context.
- System decomposition, data modelling and API boundary design.
- Technical writing, comparative evaluation, verification planning and evidence-based judgement.
- Security-minded thinking about roles, validation, auditability and data minimisation.
- Project planning, prioritisation and communication of limitations.

Individual Reflection — Athavan K (192524036)

The most difficult engineering problem was combining urgency, distance and recipient suitability without reducing the decision to a single simplistic rule. I addressed it by defining eligibility first and then using a transparent weighted score whose components can be shown to users. A key personal design decision was to delay machine-learning prediction until representative historical data exists; this protects explainability and avoids presenting an unvalidated model as reliable. I independently developed a clearer understanding of cloud service boundaries, role-based access and verification planning. If the project were repeated, I would build a small executable prototype earlier so that concurrency, notification timing and usability assumptions could be tested before finalising the architecture.

REFERENCES

- [1] B. Borusiak and B. Knežević, “Surplus food redistribution systems as a food waste prevention tool,” in *Sustainable Food: Production and Consumption Perspectives*, K. Pawlak-Lemańska, B. Borusiak and E. Sikorska, Eds. Poznań: Poznań University of Economics and Business Press, 2024, pp. 184–197, doi: 10.18559/978-83-8211-209-2/12.
- [2] P. Amrutkar, S. Bondarde, A. Kale, N. Patil and S. Habbu, “A food redistribution platform for sustainable food waste management and community empowerment,” *International Journal for Research in Applied Science & Engineering Technology*, 2024, doi: 10.22214/ijraset.2024.65493.
- [3] Z. R. Shi, L. Lizarondo and F. Fang, “A recommender system for crowdsourcing food rescue platforms,” in *Proceedings of the Web Conference 2021*, 2021, pp. 857–865, doi: 10.1145/3442381.3449787.
- [4] M. Sammer et al., “AI-FEED: Prototyping an AI-powered platform for the food charity ecosystem,” *International Journal of Computational Intelligence Systems*, vol. 17, art. 259, 2024, doi: 10.1007/s44196-024-00656-9.
- [5] Y. Yang, R. An, C. Fang and D. Ferris, “Artificial intelligence in food bank and pantry services: A systematic review,” *Nutrients*, vol. 17, no. 9, art. 1461, 2025, doi: 10.3390/nu17091461.
- [6] C. Agmapisarn, P. Seetha and P. Fakfare, “Optimizing surplus food redistribution: Leveraging digital platforms to advance sustainable consumption,” *Sustainable Development*, vol. 33, no. 3, pp. 3831–3846, 2025.
- [7] M. Wyskwarski, A. Musioł-Urbańczyk, B. Sorychta-Wojczyk and D. Zdonek, “Understanding user perceptions of food-related applications: Insights from topic modeling on food waste reduction and sustainability,” *Sustainability*, vol. 17, no. 10, 2025.
- [8] H. Tawfik, “Extending UTAUT2 to examine the utilization of mobile food-sharing apps and online platforms for the reduction of household food waste,” *Technological Sustainability*, vol. 3, no. 4, pp. 392–415, 2024.
- [9] React Documentation, “React,” [Online]. Available: <https://react.dev/>. Accessed: Sep. 3, 2026.
- [10] Node.js Documentation, “Node.js,” [Online]. Available: <https://nodejs.org/en/docs/>. Accessed: Sep. 3, 2026.
- [11] MongoDB, “MongoDB Atlas Documentation,” [Online]. Available: <https://www.mongodb.com/docs/atlas/>. Accessed: Sep. 3, 2026.
- [12] Google, “Google Maps Platform Documentation,” [Online]. Available: <https://developers.google.com/maps>. Accessed: Sep. 3, 2026.
- [13] OWASP Foundation, “Application Security Verification Standard 4.0.3,” 2021. [Online]. Available: <https://owasp.org/www-project-application-security-verification-standard/>.
- [14] OWASP Foundation, “OWASP Top 10: 2021,” 2021. [Online]. Available: <https://owasp.org/Top10/>.

- [15] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.2,” 2023. [Online]. Available: <https://www.w3.org/TR/WCAG22/>.
- [16] Government of India, “Digital Personal Data Protection Act, 2023,” Act No. 22 of 2023.
- [17] Food Safety and Standards Authority of India, “Food Safety and Standards regulations and guidance,” [Online]. Available: <https://www.fssai.gov.in/>.

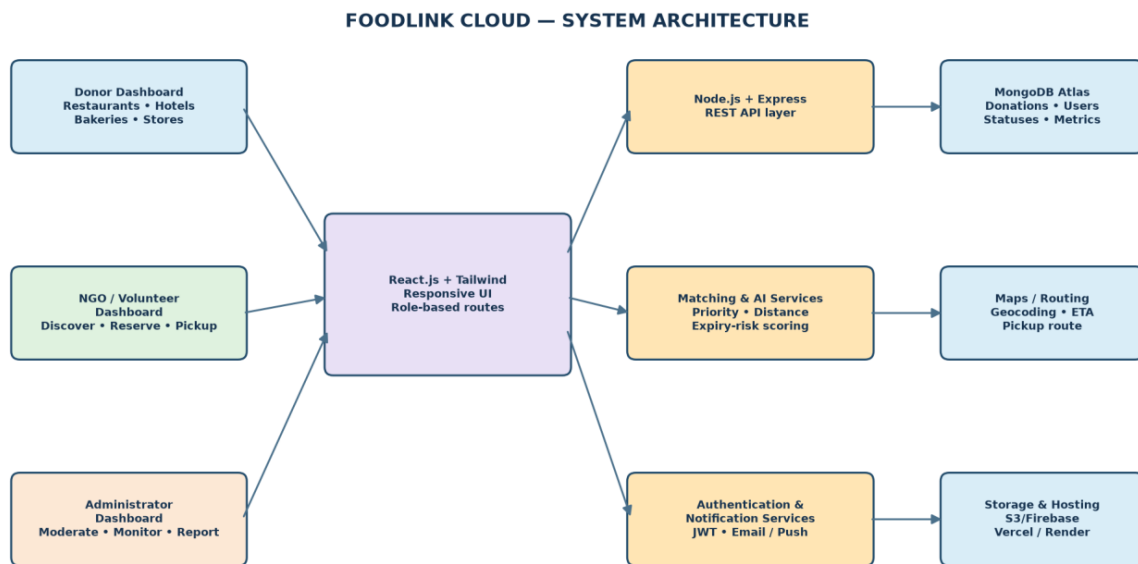
APPENDICES

Appendix A – Detailed Calculations

Illustrative priority-score calculation: assume a donation has urgency $U = 90$, distance suitability $D = 80$, capacity fit $C = 100$ and recipient reliability $R = 70$. Then $S = 0.40(90) + 0.25(80) + 0.20(100) + 0.15(70) = 36 + 20 + 20 + 10.5 = 86.5$ out of 100. This example demonstrates the calculation only; it is not a measured field result.

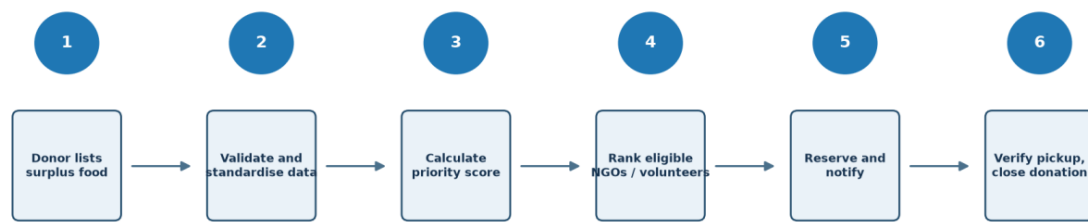
Component	Normalised value	Weight	Contribution
Urgency (U)	90	0.40	36.0
Distance suitability (D)	80	0.25	20.0
Capacity fit (C)	100	0.20	20.0
Reliability (R)	70	0.15	10.5
Total score	—	1.00	86.5 / 100

Appendix B – Engineering Drawings / Schematics



Appendix B.1 – System architecture schematic

FOODLINK CLOUD — DONATION MATCHING WORKFLOW



Every status change is recorded so donors, recipients and administrators share one source of truth.

Appendix B.2 – Donation workflow schematic

Appendix C – Source Code / Code Repository Information

The earlier report names GitHub as the intended version-control platform but does not supply a repository URL. The implementation should be maintained in an approved private or institutional repository. Only essential excerpts should be copied into the final academic report; complete source code should remain in the approved repository.

Appendix D – Datasheets

Not applicable to the software-only capstone design. Service documentation for React, Node.js, MongoDB Atlas and Google Maps Platform is listed in References [9]–[12].

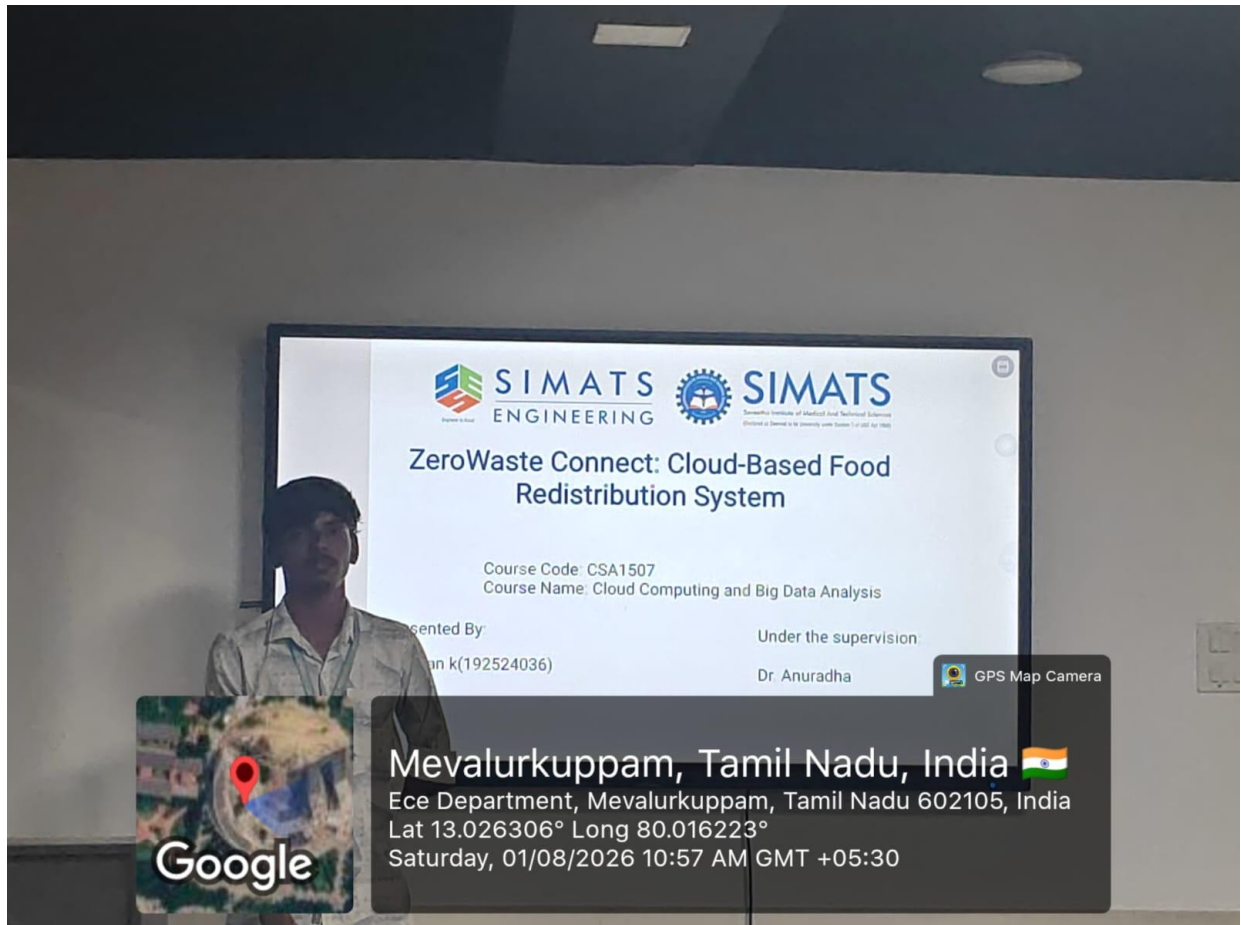
Appendix E – Experimental / Raw Data

No experimental field dataset was supplied with the earlier report. The report therefore uses representative scenario descriptions and clearly labels them as design-level verification. A future pilot should preserve anonymised donation, reservation, pickup and expiry records with timestamps.

Appendix F – Ethics / Institutional Approval

No human-subject study or beneficiary data collection is claimed in the supplied report. If a future pilot collects participant feedback or identifiable operational data, institutional approval, consent, data minimisation and retention procedures should be obtained before collection.

Appendix G – Project Meeting / Progress Records



Appendix H – User / Industry Feedback

No user or industry feedback records were supplied. Partner feedback should be collected in a future pilot to validate terminology, safe-use fields, notification timing and the usability of the reservation flow.

Appendix I – Publications / Patents / Competitions / Product Outcomes

No publication, patent, competition or product-outcome evidence was supplied for this report.

Appendix J – Individual Contribution Evidence

Evidence Area	Student	Evidence Description
Requirements and literature	Athavan K	Requirement analysis, literature synthesis and problem formulation.
Design and development	Athavan K	Architecture, data flow, matching score and interface design.
Testing and analysis	Athavan K	Verification matrix, limitations and engineering judgement.
Report and presentation	Athavan K	All sections, figures, tables, references and appendices.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed and signed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8); individual project context
Experimentation, analysis, engineering judgement	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4.8 + Appendix J